SafeCharge-RL

Reinforcement learning for EV charging when drivers' departure times are wrong or misreported

Code walkthrough: what each file does, in what order to read it, and how to answer questions about it.

Team 32	Adithyan R (AM.SC.U4AIE23065), Adithya Rajesh (AM.SC.U4AIE23007), Balu M Krishna (AM.SC.U4AIE23026)
Guide	Dr. Geetha M, Dept. of CSE (AI), Amrita Vishwa Vidyapeetham
Prepared	9 Oct 2026, for the mid-review (9-10 Oct). Code: github.com/adithyx-0/safecharge-rl (private repo)

How to use this. Section 1 says which files to open and in which order. Sections 3-6 explain every file. Section 7 traces one environment step and one training update line by line. Section 8 gives short answers to likely code questions. Section 9 lists the demo commands. All code is in `src/safecharge/` (about 890 lines), `scripts/`, `tests/` (about 420 lines, 108 tests) and `configs/`.

1. Which files to look at (read in this order)

Reality check on the '50% of code' requirement: everything the design calls Phase 0 to Phase 3 is written and run, and E4 is partly done. Only E5, the unprojected-RL baseline and SAC-Lagrangian are missing. So you are well past 50%; say 'core implementation complete, experiments running' and do not quote a percentage you cannot defend (our estimate is 80-85%).

Priority	File	Lines	Why / how long
A: know it cold	src/safecharge/env.py	116	The simulator = the MDP. Open this first. 15 min
A	src/safecharge/projection.py	30	The safety layer, the 'hard guarantee'. 5 min
A	src/safecharge/report_model.py	49	How wrong or fake reports are generated. 5 min
A	src/safecharge/agents/ppo_lag.py	209	The learning algorithm (PPO + Lagrange multipliers). 20 min
A	src/safecharge/agents/deepsets.py	105	The neural network. 10 min
A	src/safecharge/baselines.py + offline_lp.py	27 + 41	The comparison methods. 10 min
A	src/safecharge/metrics.py (paired_gain)	101	How results and the cheating gain are measured. 10 min
B: know what it does	src/safecharge/data.py, config.py, mpc.py	138 + 32 + 41	Data cleaning, config loading, MPC baseline. 10 min
B	scripts/train.py, run_experiment.py, run_paired_replay.py	81 + 119 + 108	How training and the experiments are run. 10 min
B	configs/e1.yaml, base.yaml, derived_caltech_kappa0.2.yaml	33 + 13 + 11	Every setting in one place. 5 min
C: just know they exist	scripts/download*, audit_data, calibrate*, check_uncontrolled, analyze_recorded_reports, make_tables, run_*_suite.sh, demo.py	-	Support scripts. 5 min
C	tests/ (7 files)	420	Proof it works: 108 tests. Know the headline ones. 5 min

Suggested ownership so each of you can answer on part of it: **Member 1** data.py, config.py, report_model.py, download and audit scripts. **Member 2** env.py, projection.py, deepsets.py, ppo_lag.py, train.py. **Member 3** baselines.py, offline_lp.py, mpc.py, metrics.py, run_experiment.py, run_paired_replay.py, tests.

2. How the pieces connect

Step	What happens	Code
1	Raw ACN-Data JSON becomes a list of Day objects (arrival step, departure step, energy, price) on a 15-minute grid	data.py: load_sessions, build_days
2	For each day a report model makes the 'reported departure' the policy will see (truthful, noisy, late, fake-early)	report_model.py: reports()
3	The environment plays the day step by step. It shows the policy only reported times	env.py: reset, view, _obs, step
4	The policy (or a rule, or MPC) proposes power per port; the safety layer cuts it to the grid cap	agents/*, baselines.py, mpc.py, projection.py
5	After each step: energy delivered, cost, unmet energy at the TRUE departure, wear proxy	env.py: step; metrics.py
6	Training: 48 days per update, PPO update, then the Lagrange multipliers are adjusted	agents/ppo_lag.py, scripts/train.py
7	Evaluation: same days and same report draws for every method, compare with the hindsight LP, bootstrap CIs	scripts/run_experiment.py, offline_lp.py, metrics.py
8	Cheating test: replay one day twice, honest vs 's hours early', compare energy received	metrics.py: paired_gain; scripts/run_paired_replay.py

3. The core files, one by one

`src/safecharge/data.py` **(138 lines)**

Turns recorded charging sessions into 'days' the simulator can replay.

Part	What it does
`constants T, DELTA, P_PORT, N_PORTS`	96 steps per day, 0.25 h per step, 6.656 kW per port, 54 (Caltech) / 52 (JPL) ports
`price_curve(date, tariff)`	$/kWh for each of the 96 steps. tariff='summer' uses the summer weekday curve on every day. Uses searchsorted to map each hour to its price period
`class Day`	One day: arr (first step present), dep (first step gone), energy e_i, price(96), session ids, recorded kWh, recorded driver-entered departure (rep_rec)
`load_sessions(path)`	Reads the JSON, converts connect/disconnect UTC times to LOCAL time using each session's timezone, parses the driver-entered requestedDeparture
`build_days(df, midnight, tariff, ...)`	The cleaning pipeline: drop excluded station, zero energy, bad ranges, midnight crossers; arrival rounded UP and departure rounded DOWN to 15 min; e_i clipped to what a port can deliver in the stay; weekdays only; drop daylight-saving days; group by date. Returns days + a stats dict counting every exclusion
`is_dst_day, peak_concurrency`	Helpers: clock-change check, and the max number of cars at once (used to check we never need more than N ports)

Key idea: Everything downstream sees a clean, conservative, local-time version of the real sessions, and every exclusion is counted so the share affected can be reported.

If she asks: Why round arrival up and departure down? So the window we simulate is never longer than the real stay. Why drop midnight sessions? Clipping them would invent an earlier departure, which is the very quantity we study.

`src/safecharge/report_model.py` **(49 lines)**

Creates the reported departure time for every vehicle of a day, for each report mode.

Part	What it does
`base_draws(seed, day_index, n)`	Four arrays of uniform random numbers (eps, late, sel, imp) from a generator seeded by (seed, day). They do not depend on the mode or on w, q, rho
`empirical_errors(days)`	Pool of real recorded errors (recorded minus true departure) used to fill in missing recorded reports in M1r
`reports(day, mode, ...)`	M0: copy of true departure. M2: dep + eps*w (eps in [-1,1], so error in [-w,+w] hours). M3: with probability q add up to w hours (late). M4: with probability rho report s hours EARLIER. M1r: recorded departure, else resampled real error

Part	What it does
`focal / focal_s`	Paired-replay hook: forces one vehicle to report dep minus focal_s hours
`last line np.maximum(rep, arr)`	A report is never earlier than the arrival time

Key idea: Because the random numbers are fixed per (seed, day), every method sees identical reports, and a larger w or rho reuses the same numbers (the lies are nested). That makes comparisons fair and paired.

If she asks: Where do the units come from? Reports are in 15-minute steps; w in hours is divided by DELTA = 0.25 to convert.

`src/safecharge/projection.py` (30 lines)

The safety layer: shrink requested powers so the total never exceeds the grid cap.

Part	What it does
`p0 = clip(p_tilde, 0, h)`	Requests cannot be negative or above what the vehicle can physically take
`if p0.sum() <= P_grid: return p0`	Already feasible: do nothing (identity)
`w = ones (uniform) or weights (priority)`	Uniform subtracts the same mu from everyone. Priority subtracts mu times a weight, so urgent vehicles are cut less (the E5 variant)
`bisection loop (60 iterations)`	Searches the smallest mu so that sum(clip(p_tilde - mu*w, 0, h)) is at most P_grid. The sum falls smoothly as mu grows, so bisection always works
`return clip(p_tilde - hi*w, 0, h)`	The 'hi' end of the search interval is always feasible, so the grid cap can never be violated

Key idea: A hard guarantee by construction, not by penalty. Uniform mode is the exact Euclidean projection onto the allowed set (tests compare it with a QP solver to 1e-6).

If she asks: Does the policy know about the cut? Not directly: it sees the applied total power in the next observation. Why bisection? One scalar to find, cheap (about 20-60 evaluations), no solver needed.

`src/safecharge/env.py` (116 lines)

The charging-station simulator. One episode = one day = 96 steps. It follows the Gymnasium API (reset, step).

Part	What it does
`OBS_R, OBS_TAU, OBS_PRICE`	Fixed divisors (30 kWh, 24 h, $0.27) so inputs are about unit size
`__init__(N, P_grid, P_port, e_ref, reward_scale, scheme, curtail)`	Station size, grid cap, cost normalisers, projection scheme, and the E4 'curtail' switch. Observation has 4N+4 numbers, action is N numbers in [0,1]
`reset(options={day, rep})`	Starts a day: empty ports, zero energy, plugs in whoever arrives at step 0
`_plug_in()`	Puts each arriving vehicle on the first free port; if none is free it is marked 'dropped' (never happened in our data)
`view()`	What a policy or rule may use: occupancy, remaining energy r, time left until REPORTED departure tau, laxity = tau - r/P_port, max power h. The true departure is never used here. With curtail=True, h becomes 0 after the reported departure
`_obs()`	Stacks the four per-port features and the four global ones (price, sin/cos of time of day, last total power / cap)
`step(u)`	p_tilde = u*h; p = project(...); update remaining energy and delivered energy; log power; cost in dollars; unplug vehicles whose TRUE departure is this step and record unmet energy m_i = remaining r; cost1 = unmet / e_ref; cost2 = share of ports above 80% of max power. Returns observation, reward = -dollars/reward_scale, done flag, info with costs

Key idea: The environment is where the problem is defined: the policy sees reported times, but vehicles always leave at their true time, and unmet energy is measured then. The safety projection sits inside step().

If she asks: Where is the true departure used? Only in step(), to unplug the car and count unmet energy. A test checks that changing only the true departure does not change the observation.

`src/safecharge/baselines.py` (27 lines)

The rule-based comparison methods FCFS, EDF and LLF.

Part	What it does
`rule_action(env, kind)`	Sorts occupied ports by a key: arrival time (FCFS), reported departure (EDF) or laxity (LLF). Then gives each port its maximum useful power in that order until the grid cap is used up. Returns u = p/h so the environment treats it like any action

Key idea: Rules use exactly the information the learned policy has (reported times), so the comparison is fair.

If she asks: FCFS ignores reports entirely, which is why it is unaffected by wrong reports (and also the worst on unmet energy).

`src/safecharge/offline_lp.py` (41 lines)

The 'best possible' schedule if everything about the day is known in advance: a linear program.

Part	What it does
`variables`	x[i,t] = power of vehicle i at step t, only for t between its arrival and TRUE departure; m[i] = unmet energy
`objective`	sum price(t) * x * DELTA + M * sum m. M = $5/kWh is far above the peak price $0.267, so serving energy always beats saving money
`constraints`	grid cap per step (inequality); energy balance per vehicle sum x*DELTA + m = e_i (equality); 0 <= x <= P_port
`linprog(..., method='highs')`	SciPy's HiGHS solver on sparse matrices; about 3 ms per day
`returns`	cost, unmet, objective, and the schedule (used to compute the same metrics as for other methods)

Key idea: Gives a lower bound on cost plus penalised unmet energy. The experiment script asserts, on every test day, that no method beats it. That check is our correctness test for the whole simulator.

If she asks: Why is it a lower bound? It knows the true departures and the whole day, and the other methods solve a restricted version of the same problem.

`src/safecharge/mpc.py` (41 lines)

MPC baseline: re-solve a small LP at every step using only declared deadlines.

Part	What it does
`rep_end`	Last allowed step for each plugged-in vehicle = its reported departure (at least the current step)
`LP`	Same form as the offline LP, but for vehicles currently plugged in, from now to the end of the day, with remaining energy r. No future arrivals are planned
`apply first step only`	Takes the plan's power for the current step, converts to u = p/h

Key idea: MPC is optimal if the reports are right, but it trusts them: with wrong reports it postpones charging and then runs out of time (about 30x more unmet energy at w = 2 h).

`src/safecharge/metrics.py` (101 lines)

Runs a day, computes all metrics, confidence intervals and the paired-replay cheat test.

Part	What it does
`rollout(env, day, rep, act_fn)`	Plays one full day with any policy function; returns metrics and the finished env
`day_metrics(env, ...)`	cost, kWh, cost per kWh, unmet_share = total unmet / total requested, miss_rate = share of cars below 95% of request, grid violations (count and size), high_power_ratio = share of charging port-steps above 80% of max power, Jain fairness = (sum s)^2 / (n sum s^2) with s = delivered/requested, ms per step
`_strat`	Unmet share split by length of stay (under 2 h, 2-4, 4-8, over 8 h) to check short stays are not neglected
`bootstrap_ci(x)`	Mean and 95% interval by resampling days 2000 times
`paired_gain(env, day, base_rep, act_fn, focal, s)`	Runs the same day twice: focal car honest vs reporting s hours early, everyone else identical. Gain = extra energy in the second run (kWh and % of its request)
`lp_metrics`	Same metric columns for the offline LP solution so all tables have identical columns

Key idea: One set of metric definitions for every method, and paired_gain is the measuring instrument of our main claim (can a driver gain by lying).

If she asks: Why paired? Everything else is held fixed, so any difference in energy is caused by the report alone. With s = 0 the gain is exactly zero (tested).

`src/safecharge/config.py + configs/` (32 lines + yaml files)

Loads yaml settings and returns the days of a split.

Part	What it does
`load_config(path)`	Reads yaml
`get_days(cfg, split, site)`	Builds (and caches in data/processed) the day list, then returns the days between the split's dates
`configs/base.yaml`	Site, excluded fast station, midnight='drop', tariff='summer', train/val/test/post date ranges
`configs/derived_caltech_kappa0.2.yaml`	Made by scripts/calibrate.py from TRAIN days only: P_grid 34.61 kW, P_peak 173.06, e_ref 8.359 kWh, d1 0.7133, d2 2.923, reward_scale 4.78, penalty M 5.0
`configs/e1.yaml`	Training settings: hidden size 64, learning rate 3e-4 decaying to 10%, 48 days per update, 10 epochs, minibatch 256, clip 0.2, gamma 0.999, GAE lambda 0.97, dual step 0.05, lambda cap 10, behaviour-cloning warm start from LLF, 1500 updates
`configs/e4_*.yaml`	Same, but with randomised report mixtures during training and/or the curtail environment

Key idea: Experiments differ only in config files, so any number can be traced to a config and a seed.

`src/safecharge/agents/deepsets.py` (105 lines)

The neural networks: an actor (policy) and a critic (value), both built to handle any number of cars in any order.

Part	What it does
`mlp(sizes)`	Small tanh network with orthogonal initialisation
`split_obs, pool`	Split the 4N+4 observation into per-port rows and globals; masked average of port embeddings (empty ports ignored) plus the count of occupied ports
`class Actor`	Shared encoder applied to each port -> masked mean-pool context -> shared head sees [port embedding, context, globals] and outputs a mean and a log-std for that port. Action a is Gaussian; the applied action is u = sigmoid(a). log_prob and entropy count occupied ports only
`class Critic`	Same encoder, pooled context + globals -> MLP -> 3 numbers: expected reward, expected unmet-energy cost, expected wear cost
`SelfAttn, pool_kind meanmax/attn`	Optional ablations (attention across ports; also max-pool). Off by default (the design uses mean-pool)

Key idea: Parameters are shared across ports, so the policy works for 5 cars or 45 cars and does not care about port numbering (tests check permutation equivariance and that empty ports do not change anything).

If she asks: Why Deep Sets? Number of vehicles changes constantly; a plain MLP would need a fixed slot order. Why sigmoid? Actions must lie in [0,1].

`src/safecharge/agents/ppo_lag.py` (209 lines)

PPO with two Lagrange multipliers (unmet energy, wear). Single file, CleanRL style.

Part	What it does
`gae(rew, val, gamma, lam)`	Generalised advantage estimation, computed backwards through the 96 steps
`dual_step(lam, J, d, alpha, ...)`	lambda = clip(lambda + alpha * clip(J/d - 1, -1, 1), 0, lam_max). Raise a multiplier when its cost J is above budget d, lower it when below, never negative
`class PPOLag.__init__`	Creates actor, critic, optimisers, two multipliers (start 0), 48 environments, running return scales
`_episode_reports`	Picks a report mode for each training day from the config's mixture (E1: always truthful)
`collect(train_days)`	Samples 48 training days, plays 96 steps in lockstep with the current stochastic policy, stores observations, actions, log-probs, reward and both costs

Part	What it does
`update(batch)`	1) critic values, 2) GAE separately for reward and each cost, 3) normalise advantages, 4) combined advantage = (A_reward - lam1*A_cost1 - lam2*A_cost2) / (1 + lam1 + lam2), 5) standard PPO clipped loss with entropy bonus, 10 epochs, stop early if KL too big, 6) critic regression, 7) dual ascent on the smoothed episode costs
`bc_pretrain(...)`	Warm start: regress the actor's mean onto what LLF would do (behaviour cloning). Not in the submitted design, disclosed
`policy_fn, evaluate`	Deterministic (mean) action for testing; evaluate runs a list of days
`state / load`	Checkpoint everything so training can resume

Key idea: The multipliers make the learned policy care more about a constraint exactly when it is being violated. The design's two constraints are unmet energy (budget 2% of requested) and wear (budget from baselines). The grid cap is not a constraint here: the projection handles it.

If she asks: Why not a library? OmniSafe was checked; it supports one cost and pulls many dependencies, and we need two costs plus a custom network. Why behaviour cloning? From scratch, learning is slow because the unmet-energy signal only arrives at departure.

4. Scripts

Script	What it does
train.py	Trains one seed: loads config, optional behaviour-cloning warm start, loop of collect+update; every 20 updates evaluates on validation days and saves best.pt (feasible first, then cheapest) and last.pt; writes log.csv; resumable
run_experiment.py	Evaluates FCFS, EDF, LLF, MPC, OPT and all trained seeds on one split and one report mode with identical days and draws; asserts the offline lower bound every day; writes per-day and summary CSVs with 95% bootstrap CIs and gap to optimum
run_paired_replay.py	E3/E4: for 5 random focal vehicles per day (stay of at least 2 h) and each s in 0.5, 1, 2 h, runs paired_gain for EDF, LLF, MPC and the learned policy; summarises gain with bootstrap CI over days and the share of vehicles that gain
run_eval_suite.sh	Runs the whole evaluation for one trained config: E1 (3 grid caps), E2 (M2, M3, M1r), E6 (post-COVID, JPL), E3 (paired replay)
run_e4_suite.sh	Same idea for the E4 variants
make_tables.py	Collects all summaries into uniform long tables and pivots in results/tables/
calibrate.py	From TRAIN days only: P_peak, kappa sweep, e_ref, d1, d2, reward scale; writes the derived config
calibrate_kappa_reports.py, calibrate_transfer_site.py	Where do wrong reports matter (kappa sweep with all report modes); JPL's kappa by the same rule
audit_data.py, check_uncontrolled.py	Data counts per month, midnight crossings, clipped energy; uncontrolled replay check that simulated energy matches recorded kWh (1.2% short after excluding the fast station)
analyze_recorded_reports.py	How wrong are the recorded driver-entered departures (mean error 3.15 h Caltech, 2.35 h JPL)
download_acndata_windowed.py	Month-by-month download, each month checked against the API's own total, resumable. (download_acndata.py is the older offset-based version that kept failing)
demo.py	The live demo for the presentation (new): one busy day, all methods, plus a cheat test; optional plot
make_briefing_pdfs.py, make_presentation_pdfs.py	Generate these PDFs from the result CSVs

5. Tests (108, all passing; run in 4 seconds)

File	What it proves
test_projection.py	Always within bounds and under the cap; unchanged when already feasible; applying it twice changes nothing; matches a general QP solver on random cases; priority variant feasible
test_env.py	No vehicle gets more than e_i and none charges outside its window; true departure never in the observation; observation size 4N+4; uncontrolled run delivers everything; offline LP is a lower bound for rules and random policies; paired replay with s=0 gives exactly 0; same seed gives identical results; curtail stops service at the reported time and removes the cheating gain for EDF
test_report_model.py	M0 truthful; same seed same draws; reports not before arrival; M2 within bounds and nested in w; M3 only late; M4 nested in rho; focal shift of 0 is identity; M1r deterministic
test_deepsets.py	Permuting ports permutes the policy output (and leaves the critic unchanged); empty ports have no effect; log-prob equals torch Normal on occupied ports; works with different car counts
test_ppo_lag.py	GAE equals hand computation; dual ascent rises when violated, falls when satisfied, never below 0; clipping and cap work
test_mpc.py	MPC is feasible and never beats the offline optimum; single-vehicle case equals the offline optimum

Note on test strength: deliberately halving the LP grid cap in the code made 13 tests fail, so the tests do catch real errors.

6. Walk-through: one environment step

#	Line of thinking in step(u)	Example
1	view() gives h = max useful power per port	6.656 kW if the car needs more than 1.66 kWh left, less when nearly full
2	p_tilde = clip(u,0,1) * h	Policy asks u = 0.8 on a car with h = 6.656: wants 5.3 kW
3	p = project(p_tilde, h, P_grid)	If all requests add to 60 kW and the cap is 34.6 kW, every request is cut by the same mu
4	Remaining energy r -= p*DELTA; delivered E += p*DELTA	5.3 kW for 15 min = 1.33 kWh
5	dollars = price * total power * DELTA; reward = -dollars / reward_scale	At peak price this step is expensive
6	Cars whose TRUE departure is this step: m_i = r; cost1 += m_i; port freed	A car leaves still needing 2 kWh: cost1 += 2/e_ref
7	cost2 = (number of ports with p above 80% of max) / N	Wear proxy
8	t += 1; plug in new arrivals; return observation, reward, done, info	Done after step 96

7. Walk-through: one PPO-Lagrangian update

- **collect:** 48 random training days x 96 steps with the current policy (about 4,600 samples).
- **critic and GAE:** three value heads (reward, unmet cost, wear cost) give three advantage signals.
- **combine:** advantage = (A_reward - lambda1*A_unmet - lambda2*A_wear) / (1 + lambda1 + lambda2). A large lambda makes the policy treat that cost like a big penalty.
- **PPO step:** maximise min(ratio*adv, clip(ratio, 0.8, 1.2)*adv) plus a small entropy bonus; 10 passes over minibatches of 256; gradient clipping 0.5; stop early if the policy moved too far (KL).
- **dual step:** average undiscounted cost per day J (smoothed). If J above budget d, lambda goes up by at most 0.05 per update; if below, down; capped at 10.
- **repeat 1,500 times** (about 35-40 min per seed on the laptop). Every 20 updates: evaluate on validation days and save the best feasible checkpoint.

8. Likely code questions and short answers

Question	Answer
How do you make sure the grid limit is never exceeded?	projection.py runs inside env.step. The bisection returns the feasible end of its interval, so the sum is at most P_grid every step. 0 violations in all runs; a test compares it with a QP solver.
Does the policy see the true departure?	No. view() and _obs() only use the reported time. step() uses the true time only to unplug and to count unmet energy. A test enforces this.
How do you compare methods fairly?	Same test days and same report draws (fixed seed 12345), same metric code, and the offline LP lower bound is asserted on every day.
What are the two cost functions?	cost1 = unmet energy at departure / reference energy; cost2 = share of ports above 80% of max power, our wear proxy since ACN-Data has no battery data.
How are the Lagrange multipliers updated?	dual_step: lambda increases when the smoothed daily cost exceeds its budget, decreases otherwise, clipped to [0, 10].
Why the behaviour-cloning warm start?	Without it the policy stayed worse than every rule after 2,000 updates. It is not in the submitted design and we say so.
Why is gamma 0.999 in the config and 0.99 in the design?	With 96 steps, 0.99 discounts early actions to about 38% by day end, while the unmet-energy cost arrives at departure. We chose 0.999 on validation and disclose it.
How is the data split?	Chronological: train Jan-Aug 2019, validation Sep-Oct 2019, test Nov 2019-Feb 2020. Constants like P_grid are computed from train days only.
What does paired replay do?	Two runs of the same day with the same policy: focal car honest vs s hours early. The difference in its delivered energy is the misreporting gain.
Why a custom simulator and not ACN-Sim?	ACN-Sim works in amps and network matrices and its data client failed authentication; the design allowed a thin Gymnasium wrapper as fallback. Our numpy env runs about 32,000 steps/s.
Where could a result come from by chance?	5 trained seeds, 85 test days, bootstrap 95% CIs over days. E4 has only 3 seeds.
What would you do next in code?	Finish the 3 paused E4 runs, run E5 (priority projection, no projection with a soft penalty), optional SAC-Lagrangian.
What is not finished?	E5 on the test split, the unprojected-RL baseline (not implemented), SAC-Lagrangian, E4 seed 2 runs.

9. Demo commands (from the repo root)

Command	Shows	Time
`.venv/bin/python -m pytest tests -q`	108 passed	4 s
`.venv/bin/python scripts/demo.py 58 --plot`	Busy day (23 Jan 2020, 39 cars): table of all methods. The learned policy has a high-power ratio of 0.12 vs 0.75, and the plot results/demo_day.png shows power sitting at the grid cap with no violation	4 s
`.venv/bin/python scripts/demo.py 7`	Crowded day (12 Nov 2019): reporting 1-2 h early gains extra energy: EDF +9.4%, LLF +10.4%, learned +15.7% for one of the three tested cars	4 s
`open results/tables/e1_m0_E1_long.csv`	The result table behind the E1 slide	-
`tail results/runs/e1_m0/seed0/log.csv`	Training log (unmet energy, cost, lambda per update)	-

Rehearse the demo once before the review. The demo needs the trained checkpoint `results/runs/e1_m0/seed0/best.pt` on the presenting laptop (it is not in GitHub).